

실습 #10

Lab10_1

Lab10_1.java 코드

```
import java.util.Random;

public class Lab10_1 {
    public static void main(String[] args) {
        Dish d = new Dish();
        Producer cheeseProducer = new Producer(Hamburger.CHEEZE, d);
        Producer fishProducer = new Producer(Hamburger.FISH, d);
        Consumer c1 = new Consumer("철수", d);
        Consumer c2 = new Consumer("민수", d);

        cheeseProducer.start();
        fishProducer.start();
        c1.start();
        c2.start();
    }
}

class Hamburger {
    public static final int CHEEZE = 0;
    public static final int FISH = 1;
    static int serial_number = 0;

    public int kind;
    private int id;

    Hamburger(int kind) {
        this.kind = kind;
        this.id = serial_number++;
    }
}
```

```

    public String toString() {
        if (kind == CHEEZE)
            return ("치즈버거" + id);
        else
            return ("피시버거" + id);
    }
}

class Producer extends Thread {
    int kind;
    Dish dish;
    Random r = new Random();

    Producer(int kind, Dish d) {
        this.kind = kind;
        this.dish = d;
    }

    private Hamburger produce() {
        Hamburger h = new Hamburger(kind);
        dish.add(h);
        return h;
    }

    public void run() {
        while (true) {
            try {
                sleep(r.nextInt(5000));
            } catch (Exception e) {
            }
            Hamburger h = produce();
        }
    }
}

class Consumer extends Thread {
    String name;

```

```

Dish dish;
Random r = new Random();

Consumer(String name, Dish d) {
    this.name = name;
    this.dish = d;
}

private Hamburger consume() {
    return (dish.get());
}

public void run() {
    while (true) {
        try {
            sleep(r.nextInt(5000));
        } catch (Exception e) {
        }
        Hamburger h = consume();
    }
}

}

class Dish {
    static final int DishSize = 4;
    static final int CheezeMax = 2;
    static final int FishMax = DishSize - CheezeMax;

    int cheezenum = 0;
    int fishnum = 0;
    Hamburger[] dish = new Hamburger[DishSize];
    int fresh = -1;
    int old = -1;

    public synchronized void add(Hamburger h) {
        try {
            while ((h.kind == Hamburger.CHEEZE && cheezenum >= CheezeMax) ||

```

```

        (h.kind == Hamburger.FISH && fishnum >= FishMax))
{
    wait();
}

if (old == -1)
    old = 0;

fresh = (fresh + 1) % DishSize;
dish[fresh] = h;

if (h.kind == Hamburger.CHEEZE) {
    cheezenum++;
} else {
    fishnum++;
}

System.out.println(h + " is added to the dish: " + this.toString());

    notifyAll();

} catch (InterruptedException e) {
    e.printStackTrace();
}
}

public synchronized Hamburger get() {
    Hamburger h = null;
    try {
        while (cheezenum == 0 && fishnum == 0) {
            wait();
        }

        h = dish[old];
        dish[old] = null;

        if (h.kind == Hamburger.CHEEZE) {

```

```

        cheezenum--;
    } else {
        fishnum--;
    }

    System.out.println(h + " is consumed from the dish: "
+ this.toString());

    old = (old + 1) % DishSize;

    notifyAll();

} catch (InterruptedException e) {
    e.printStackTrace();
}
return h;
}

public String toString() {
    return "[" + dish[0] + "," + dish[1] + "," + dish[2] +
    "," + dish[3] + "]";
}
}

```

실행 결과

```

alpha ...\OOP-class > ? main X!? v21.0.11 11:41 > java week13.lab.Lab10_1
피시버거0 is added to the dish: [피시버거0,null,null,null]
피시버거0 is consumed from the dish: [null,null,null,null]
피시버거1 is added to the dish: [null,피시버거1,null,null]
피시버거1 is consumed from the dish: [null,null,null,null]
치즈버거2 is added to the dish: [null,null,치즈버거2,null]
치즈버거2 is consumed from the dish: [null,null,null,null]
치즈버거3 is added to the dish: [null,null,null,치즈버거3]
치즈버거3 is consumed from the dish: [null,null,null,null]
치즈버거4 is added to the dish: [치즈버거4,null,null,null]
치즈버거4 is consumed from the dish: [null,null,null,null]
피시버거5 is added to the dish: [null,피시버거5,null,null]
피시버거5 is consumed from the dish: [null,null,null,null]
피시버거6 is added to the dish: [null,null,피시버거6,null]
피시버거6 is consumed from the dish: [null,null,null,null]
치즈버거7 is added to the dish: [null,null,null,치즈버거7]
치즈버거7 is consumed from the dish: [null,null,null,null]
피시버거8 is added to the dish: [피시버거8,null,null,null]
피시버거8 is consumed from the dish: [null,null,null,null]
치즈버거9 is added to the dish: [null,치즈버거9,null,null]
치즈버거9 is consumed from the dish: [null,null,null,null]
피시버거10 is added to the dish: [null,null,피시버거10,null]
피시버거11 is added to the dish: [null,null,피시버거10,피시버거11]

```

Lab10_2

Lab10_2_1.java 코드 (수정 전 코드)

```

class Spaghetti {
    private int amount;

    public Spaghetti(int amount) {
        this.amount = amount;
    }

    public void scoop() {
        amount--;
    }

    public boolean isEmpty() {
        return amount == 0;
    }
}

class Fork {
    private boolean locked;

    public Fork() {

```

```

        locked = false;
    }

    public synchronized void lock() {
        if (locked) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println("사용자에 의해 중단됨");
            }
        }
        locked = true;
    }

    public synchronized void unlock() {
        locked = false;
        notifyAll();
    }
}

public class Lab10_2_1 extends Thread {
    String name;
    Spaghetti bowl;
    Fork left, right;
    int scoops;

    public Lab10_2_1(String name, Spaghetti bowl, Fork left,
Fork right) {
        this.name = name;
        this.bowl = bowl;
        this.left = left;
        this.right = right;
        scoops = 0;
    }

    public void getForks() {
        left.lock();
        right.lock();
    }
}

```

```

}

public void releaseForks() {
    right.unlock();
    left.unlock();
}

public void eat() {
    getForks();
    bowl.scoop();
    scoops++;
    print();
    releaseForks();
}

public void print() {
    System.out.printf("%s: %d 포크 먹음%n", name, scoops);
}

public void finalPrint() {
    System.out.print("최종적으로... ");
    print();
}

public void run() {
    while (!bowl.isEmpty()) {
        eat();
    }
}

public static void main(String args[]) {
    final int NUM_OF_GUESTS = 2;
    final int BOWL_SIZE = 5000000;
    Lab10_2_1 guest[] = new Lab10_2_1[NUM_OF_GUESTS];
    Fork fork[] = new Fork[NUM_OF_GUESTS + 1];
    Spaghetti bowl[] = new Spaghetti[NUM_OF_GUESTS];
    String names[] = { "이황", "헤겔", "공자", "이이", "장자"
};

```

```

for (int i = 0; i < NUM_OF_GUESTS; ++i) {
    fork[i] = new Fork();
    bowl[i] = new Spaghetti(BOWL_SIZE);
}
fork[NUM_OF_GUESTS] = fork[0];

for (int i = 0; i < NUM_OF_GUESTS; ++i) {
    int left = i;
    int right = i + 1;

    guest[i] = new Lab10_2_1(names[i], bowl[i], fork[left], fork[right]);
    guest[i].start();
}

for (int i = 0; i < NUM_OF_GUESTS; ++i) {
    try {
        guest[i].join();
        guest[i].finalPrint();
    } catch (InterruptedException e) {
        System.out.println("PhilosopherOriginal.main(): 사용자에 의해 중단됨");
    }
}
System.out.println("*** 식사 끝 ***");
}
}

```

실행 결과

교착 상태에 걸림

Lab10_2_2.java 코드 (수정 후 코드)

```

class Spaghetti {
    private int amount;

    public Spaghetti(int amount) {
        this.amount = amount;
    }

    public void scoop() {
        amount--;
    }

    public boolean isEmpty() {
        return amount == 0;
    }
}

class Fork {
    private boolean locked;

    public Fork() {
        locked = false;
    }

    public synchronized void lock() {
        if (locked) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println("사용자에 의해 중단됨");
            }
        }
        locked = true;
    }

    public synchronized void unlock() {
        locked = false;
        notifyAll();
    }
}

```

```

}

public class Lab10_2_2 extends Thread {
    String name;
    Spaghetti bowl;
    Fork left, right;
    int scoops;

    public Lab10_2_2(String name, Spaghetti bowl, Fork left,
Fork right) {
        this.name = name;
        this.bowl = bowl;
        this.left = left;
        this.right = right;
        scoops = 0;
    }

    public void getForks() {
        left.lock();
        right.lock();
    }

    public void releaseForks() {
        right.unlock();
        left.unlock();
    }

    public void eat() {
        getForks();
        bowl.scoop();
        scoops++;
        print();
        releaseForks();
    }

    public void print() {
        System.out.printf("%s: %d 포크 먹음%n", name, scoops);
    }
}

```

```

public void finalPrint() {
    System.out.print("최종적으로... ");
    print();
}

public void run() {
    while (!bowl.isEmpty()) {
        eat();
    }
}

public static void main(String args[]) {
    final int NUM_OF_GUESTS = 2;
    final int BOWL_SIZE = 5000000;
    Lab10_2_2 guest[] = new Lab10_2_2[NUM_OF_GUESTS];
    Fork fork[] = new Fork[NUM_OF_GUESTS + 1];
    Spaghetti bowl[] = new Spaghetti[NUM_OF_GUESTS];
    String names[] = { "이황", "헤겔", "공자", "이이", "장자"
};

    for (int i = 0; i < NUM_OF_GUESTS; ++i) {
        fork[i] = new Fork();
        bowl[i] = new Spaghetti(BOWL_SIZE);
    }
    fork[NUM_OF_GUESTS] = fork[0];

    for (int i = 0; i < NUM_OF_GUESTS; ++i) {
        int left = i;
        int right = i + 1;

        if (i == NUM_OF_GUESTS - 1) {
            right = i;
            left = i + 1;
        }

        guest[i] = new Lab10_2_2(names[i], bowl[i], fork[left], fork[right]);
    }
}

```

```

        guest[i].start();
    }

    for (int i = 0; i < NUM_OF_GUESTS; ++i) {
        try {
            guest[i].join();
            guest[i].finalPrint();
        } catch (InterruptedException e) {
            System.out.println("Lab10_2_2.main(): 사용자에게 의해
중단됨");
        }
    }

    System.out.println("*** 식사 끝 ***");
}
}

```

실행 결과

```

alpha ...\OOP-class\week13\lab  main X!? v21.0.11 12:09  java Lab10_2_2
이황: 1 포크 먹음
헤겔: 1 포크 먹음
헤겔: 2 포크 먹음
헤겔: 3 포크 먹음
헤겔: 4 포크 먹음
헤겔: 5 포크 먹음
이황: 2 포크 먹음
이황: 3 포크 먹음
이황: 4 포크 먹음
이황: 5 포크 먹음
이황: 6 포크 먹음
이황: 7 포크 먹음
이황: 8 포크 먹음
이황: 9 포크 먹음
이황: 10 포크 먹음
이황: 11 포크 먹음

```

01	이화여자대학교	1050032	포크	모
01	이화여자대학교	1050033	포크	모
01	이화여자대학교	1050034	포크	모
01	이화여자대학교	1050035	포크	모
01	이화여자대학교	1050036	포크	모
01	이화여자대학교	1050037	포크	모
01	이화여자대학교	1050038	포크	모
01	이화여자대학교	1050039	포크	모
01	이화여자대학교	1050040	포크	모
01	이화여자대학교	1050041	포크	모
01	이화여자대학교	1050042	포크	모
01	이화여자대학교	1050043	포크	모
01	이화여자대학교	1050044	포크	모
01	이화여자대학교	1050045	포크	모
01	이화여자대학교	1050046	포크	모
01	이화여자대학교	1050047	포크	모
01	이화여자대학교	1050048	포크	모
01	이화여자대학교	1050049	포크	모
01	이화여자대학교	1050050	포크	모
01	이화여자대학교	1050051	포크	모
01	이화여자대학교	1050052	포크	모
01	이화여자대학교	1050053	포크	모
01	이화여자대학교	1050054	포크	모
01	이화여자대학교	1050055	포크	모
01	이화여자대학교	1050056	포크	모
01	이화여자대학교	1050057	포크	모
01	이화여자대학교	1050058	포크	모
01	이화여자대학교	1050059	포크	모
01	이화여자대학교	1050060	포크	모
01	이화여자대학교	1050061	포크	모
01	이화여자대학교	1050062	포크	모
01	이화여자대학교	1050063	포크	모
01	이화여자대학교	1050064	포크	모
01	이화여자대학교	1050065	포크	모

이후 교착 없이 정상 작동 하는 것을 확인한 후 작업을 임의로 중단함.

실행 결과의 차이

실행 결과의 차이점

원본 코드는 프로그램을 실행하면 식사가 시작 되지만 설정 된 500만번을 채우지 못하고 멈춰 버림. 이는 스레드가 교착 상태에 빠져 무한 대기 상태가 되지 때문임

반면 개선 코드는 500만번의 출력이 멈추지 않고 끝까지 진행됨. 모든 철학자가 식사를 마치고 정상 종료됨.

차이가 나는 이유

Lab10_2_1 코드는 모든 철학자가 자신의 왼쪽 포크를 먼저 집도록 설계되어 있음. 동시에 각자의 왼쪽 포크를 들게 되면, 두 사람 모두 오른쪽 포크를 얻기 위해 무한 대기하게 됨. 이것이 교착 상태임.

반면 개선된 코드에서는 마지막에 위치한 철학자가 오른쪽 포크를 먼저 집도록 순서를 반대로 변경. 자원을 요청하는 순서를 제한하여 둘 중 한 명은 포크 두 개를 모두 지고 식사를 진행할 수 있게 되어 교착 상태를 회피할 수 있음.

Lab10_3

Lab10_3_server.java 코드

```
import java.io.*;
import java.net.*;

public class Lab10_3_Server {
    public static void main(String[] args) {
        ServerSocket myService = null;
        Socket serviceClient = null;
        BufferedReader is = null;
        DataOutputStream os = null;

        try {
            myService = new ServerSocket(5000);

            serviceClient = myService.accept();
```

```

        is = new BufferedReader(new InputStreamReader(s
erviceClient.getInputStream()));
        os = new DataOutputStream(serviceClient.getOutp
utStream());

        String message = is.readLine();
        System.out.println("TCP Server:" + message);

        os.writeBytes("Hello2  from TCP Server\n");

        os.close();
        is.close();
        serviceClient.close();
        myService.close();

    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

Lab10_3_Client.java 코드

```

import java.io.*;
import java.net.*;

public class Lab10_3_Client {
    public static void main(String[] args) {
        Socket myClient = null;
        BufferedReader is = null;
        DataOutputStream os = null;

        try {
            myClient = new Socket("127.0.0.1", 5000);

            is = new BufferedReader(new InputStreamReader(m
yClient.getInputStream()));

```

```

        os = new DataOutputStream(myClient.getOutputStream());

        os.writeBytes("Hello.\n");

        String reply = is.readLine();
        System.out.println("TCP Client: " + reply);

        os.close();
        is.close();
        myClient.close();

    } catch (UnknownHostException uhe) {
        uhe.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

실행 결과

The image shows two side-by-side terminal windows. The left window displays the output of a Java TCP server, showing 'TCP Server: Hello.' and a timestamp of 12:45. The right window displays the output of a Java TCP client, showing 'TCP Client: Hello2 from TCP Server' and a timestamp of 12:43. Both windows show the command 'java week13.lab.Lab10_3_Server' and 'java week13.lab.Lab10_3_Client' respectively.